

New Uzbekistan University

Department of Computer Science & Software Engineering

LAB 2: DART PROGRAMMING WORKBOOK

Comprehensive Core Concepts & Problem Set

Course Code: SE202

Course Title: Mobile Programming

Assignment: Lab 2 Workbook

Language: Dart 3.x

Professor: Mukhkammadali Khayotov

Student Name: _____

Student ID: _____

Date: September 20, 2026

Contents

1	Main Function	1
2	Variables & Data Types	1
3	Control Flow	2
4	Functions / Methods	2
5	Comments & Documentation	3
6	Classes & Constructors	3
7	Enums	4
8	Inheritance	4
9	Mixins & Interfaces	5
10	Polymorphism	6
11	Async Operations	6
12	Exceptions & Error Handling	7

1 Main Function

Listing 1: Solution for Problem 1.1: Main Function Entry Point with Command-Line Arguments

```
1 void main(List<String> arguments) {  
2   print('Hello, Dart World!');  
3   if (arguments.isNotEmpty) {  
4     print('Command line arguments passed: ${arguments.join(", ")}');  
5   }  
6 }
```

1. **Solution Provided Above.** Write a Dart program with a **main** function entry point accepting command-line arguments and printing them.
2. Write a **main** function that prints your full name, student ID, and major on separate lines.
3. Modify the **main** function to count and display the total number of command-line arguments provided.
4. Create a **main** function that takes command-line arguments representing numbers and prints their arithmetic average.
5. Write a **main** function that validates if exactly two arguments are provided; if not, print a usage warning.
6. Create a top-level **main** function returning an exit code explicitly using **exitCode** from **dart:io**.
7. Write a **main** function supporting key-value flag parsing from raw command-line string arguments (e.g., `--port=8080`).

2 Variables & Data Types

Listing 2: Solution for Problem 2.1: Variable Declaration Modifiers

```
1 void main() {  
2   var mutableName = 'Alice';  
3   final String birthCity = 'Tashkent';  
4   const double pi = 3.14159;  
5   late String lazyDescription;  
6  
7   lazyDescription = 'Initialized later!';  
8   print('$mutableName born in $birthCity. Math constant: $pi. Status:  
9     $lazyDescription');  
}
```

1. **Solution Provided Above.** Demonstrate variable declarations using **var**, **final**, **const**, and **late**.
2. Declare variables for an integer **age**, double **gpa**, string **country**, and boolean **isStudent**.
3. Write a program demonstrating the difference between **final** and **const** using **DateTime.now()**.
4. Implement sound null safety by creating nullable (**String?**) and non-nullable variables with default fallback operators (**??**).
5. Create a program demonstrating implicit type promotion from dynamic objects to specific types.

6. Construct a record type variable storing a 3D coordinate (x, y, z) and access its positional fields.
7. Implement custom type aliases using `typedef` for a complex nested `Map<String, dynamic>` structure storing user profiles.

3 Control Flow

Listing 3: Solution for Problem 3.1: Switch Expressions & Pattern Matching

```

1 void main() {
2   int score = 85;
3   String grade = switch (score) {
4     >= 90 => 'A',
5     >= 80 => 'B',
6     >= 70 => 'C',
7     _ => 'F'
8   };
9   print('Grade achieved: $grade');
10 }

```

1. **Solution Provided Above.** Use Dart 3 switch expressions and relational patterns to evaluate grade scores.
2. Write a program that checks whether a given integer is positive, negative, or zero using an `if-else` chain.
3. Use both a standard `for` loop and a `for-in` loop to calculate the factorial of an integer.
4. Implement a `while` loop simulating a guess-the-number game that breaks upon reaching a target value.
5. Use labeled statements with `break` and `continue` to break out of nested loops.
6. Write a switch statement utilizing object pattern matching on dynamic JSON-like Map inputs.
7. Build a custom iterable state machine loop that transitions through complex application lifecycle states.

4 Functions / Methods

Listing 4: Solution for Problem 4.1: Named Parameters and Default Values

```

1 double calculateTotal(double price, {double discount = 0.0, double tax = 0.08}) {
2   double discounted = price * (1 - discount);
3   return discounted * (1 + tax);
4 }
5
6 void main() {
7   print('Total: \${calculateTotal(100.0, discount: 0.15).toStringAsFixed(2)}');
8 }

```

1. **Solution Provided Above.** Define a function using required positional and optional named parameters with default values.

2. Write an arrow syntax function `bool isEven(int n)` returning true if a number is even.
3. Create a function taking optional positional parameters `[String prefix, String suffix]`.
4. Write a higher-order function accepting a list of integers and a transformer callback function.
5. Implement a recursive function to compute the N -th Fibonacci number.
6. Create an anonymous closure that maintains a private counter variable across function invocations.
7. Implement a generic function signature with type constraints `T extends Comparable<T>`.

5 Comments & Documentation

Listing 5: Solution for Problem 5.1: Dartdoc Comments and Annotations

```

1  /// Represents a bank account balance.
2  ///
3  /// Throws an [ArgumentError] if [initialDeposit] is negative.
4  class BankAccount {
5    double balance;
6    BankAccount(double initialDeposit) : balance = initialDeposit {
7      if (initialDeposit < 0) throw ArgumentError('Deposit cannot be negative');
8    }
9  }

```

1. **Solution Provided Above.** Write a class with full Dartdoc documentation comments and brackets cross-referencing parameters.
2. Add single-line (`///`) and multi-line (`/* */`) comments explaining a mathematical calculation.
3. Write Dartdoc comments for a data validation utility class specifying parameters, return types, and exceptions.
4. Use Markdown formatting inside a Dartdoc comment including bold text, bullet points, and code blocks.
5. Annotate code elements using `@deprecated` and `@override` accompanied by explaining doc comments.
6. Build a fully documented API class structure ready for generation via the `dart doc` command.
7. Enforce strict documentation linter rules by configuring custom `analysis_options.yaml` rules.

6 Classes & Constructors

Listing 6: Solution for Problem 6.1: Factory, Named, and Const Constructors

```

1  class Point {
2    final double x, y;
3    const Point(this.x, this.y);
4    Point.origin() : x = 0, y = 0;
5    factory Point.fromJson(Map<String, double> json) =>
6      Point(json['x'] ?? 0, json['y'] ?? 0);
7  }

```

1. **Solution Provided Above.** Construct a class with default, named, constant, and factory constructors.
2. Create a `Person` class with fields for name and age, plus a standard constructor.
3. Implement an initializer list in a constructor to validate input boundaries prior to field assignment.
4. Build a thread-safe Singleton pattern class using a private constructor and static factory constructor.
5. Implement custom getter and setter properties enforcing domain constraints.
6. Create an immutable data transfer class using `const` constructors and final field modifiers.
7. Design a class with redirecting constructors delegating tasks to a primary master constructor.

7 Enums

Listing 7: Solution for Problem 7.1: Enhanced Enums with Fields and Getters

```

1 enum Planet {
2     mercury(mass: 3.3e23),
3     venus(mass: 4.87e24),
4     earth(mass: 5.97e24);
5
6     final double mass;
7     const Planet({required this.mass});
8     bool get isHabitable => this == Planet.earth;
9 }

```

1. **Solution Provided Above.** Define an enhanced enum containing fields, constructor arguments, and custom getters.
2. Declare a basic enum `Day` containing days of the week and iterate over all values using `Day.values`.
3. Write a function mapping an enum value to a UI display string using a switch expression.
4. Create an enhanced enum that implements an abstract interface and defines computed methods.
5. Parse raw string data safely into enum values using `Enum.valuesByName()`.
6. Implement an enhanced enum with generic parameters and static helper methods.
7. Implement a state-machine transition manager driven completely by enhanced enum logic.

8 Inheritance

Listing 8: Solution for Problem 8.1: Class Extension and Method Overriding

```

1 class Vehicle {
2     final String brand;
3     Vehicle(this.brand);
4     void start() => print('$brand starting...');
5 }

```

```

6
7 class ElectricCar extends Vehicle {
8   final int batteryCapacity;
9   ElectricCar(String brand, this.batteryCapacity) : super(brand);
10
11   @override
12   void start() {
13     super.start();
14     print('Battery level: $batteryCapacity kWh');
15   }
16 }

```

1. **Solution Provided Above.** Demonstrate class extension, super constructors, and method overriding.
2. Create a base class **Animal** and a derived class **Dog** overriding a **makeSound()** method.
3. Demonstrate super-initializer parameters syntax (**ElectricCar(super.brand)**).
4. Build a multi-level inheritance hierarchy (**Shape** → **Polygon** → **Triangle**).
5. Create an abstract base class with concrete and abstract methods enforced on subclasses.
6. Prevent class subclassing and overriding using Dart 3 **final** and **base** class modifiers.
7. Construct a complex hierarchy passing immutable configuration parameters up the inheritance chain.

9 Mixins & Interfaces

Listing 9: Solution for Problem 9.1: Mixin Composition and Implicit Interfaces

```

1 abstract class Printable { void printData(); }
2
3 mixin TimestampLogger on Printable {
4   void logWithTime() {
5     print('${DateTime.now()}:');
6     printData();
7   }
8 }
9
10 class Report implements Printable with TimestampLogger {
11   @override
12   void printData() => print('Q3 Financial Summary');
13 }

```

1. **Solution Provided Above.** Implement an implicit interface and apply a constrained mixin via the **with** keyword.
2. Create an interface class **DBConnector** and implement it in a concrete **MySQLConnector** class.
3. Create a mixin **Flyable** with a method **fly()** and apply it to a **Bird** class.
4. Combine multiple mixins (**Walker**, **Swimmer**, **Flyable**) onto a single **Duck** class.
5. Restrict mixin application to specific superclasses using the **on** keyword.
6. Compare interface implementation (**implements**) vs mixin composition (**with**) via code examples.

7. Design a modular Flutter-style state management architecture using mixin compositions.

10 Polymorphism

Listing 10: Solution for Problem 10.1: Dynamic Interface Dispatch

```
1 abstract class PaymentProcessor { void process(double amount); }
2
3 class CreditCardProcessor implements PaymentProcessor {
4   @override void process(double amount) => print('Paid \$$amount via Credit Card');
5 }
6
7 class CryptoProcessor implements PaymentProcessor {
8   @override void process(double amount) => print('Paid \$$amount via Crypto Wallet')
9   ;
10 }
11 void checkout(PaymentProcessor p, double amt) => p.process(amt);
```

1. **Solution Provided Above.** Demonstrate runtime polymorphic dispatch using abstract interface classes.
2. Create a list of `Shape` objects (`Circle`, `Rectangle`) and call `area()` on each polymorphically.
3. Demonstrate runtime type checks using `is` and type casting using `as`.
4. Implement parametric polymorphism using generic classes `Repository<T>`.
5. Use Dart 3 sealed classes to construct exhaustive type matching patterns.
6. Implement the Strategy Design Pattern using polymorphic interface delegates.
7. Build a plugin system where dynamic handlers process incoming actions polymorphically.

11 Async Operations

Listing 11: Solution for Problem 11.1: Asynchronous Futures and Stream Generators

```
1 Future<String> fetchUser() async {
2   await Future.delayed(const Duration(seconds: 1));
3   return 'User #1024';
4 }
5
6 Stream<int> countStream(int max) async* {
7   for (int i = 1; i <= max; i++) {
8     await Future.delayed(const Duration(milliseconds: 200));
9     yield i;
10  }
11 }
```

1. **Solution Provided Above.** Implement asynchronous functions using `Future`, `async/await`, and generator `Stream`.
2. Write an `async` function simulating a database lookup returning user data after a 2-second delay.
3. Use `Future.wait()` to run three asynchronous tasks concurrently and aggregate their results.

4. Create a stream subscriber that listens to periodic timer ticks and cancels after 5 emissions.
5. Transform stream values using `map()`, `where()`, and `distinct()` operators.
6. Implement error handling on an asynchronous stream pipeline using `handleError` callbacks.
7. Build an event-driven broadcast stream controller that manages multi-subscriber real-time updates.

12 Exceptions & Error Handling

Listing 12: Solution for Problem 12.1: Custom Exception and Exception Handling

```

1 class InsufficientFundsException implements Exception {
2     final double required;
3     InsufficientFundsException(this.required);
4     @Override String toString() => 'InsufficientFundsException: Missing \$$required';
5 }
6
7 void withdraw(double amount, double balance) {
8     if (amount > balance) throw InsufficientFundsException(amount - balance);
9 }
10
11 void main() {
12     try {
13         withdraw(150, 100);
14     } on InsufficientFundsException catch (e, stackTrace) {
15         print('Caught custom exception: $e');
16     } finally {
17         print('Transaction complete.');
```

1. **Solution Provided Above.** Implement a custom exception class and handle errors using `try-on-catch-finally`.
2. Write a function dividing two numbers that catches `UnsupportedError` when dividing by zero.
3. Throw an `ArgumentError` if an incoming string parameter is empty or null.
4. Use the `on` clause to handle specific exception types differently from generic exceptions.
5. Capture and print full execution stack traces during debugging scenarios.
6. Implement an exception rethrow mechanism using the `rethrow` keyword.
7. Build a robust API client wrapper converting raw network exceptions into structured domain failure objects.